PUC Minas



Aula 10 · Front-end

Context API e hooks avançados

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Identificar o problema do **prop drilling**
2. Criar um **Context** com Provider e hook de acesso
3. Usar `useReducer` para estados com muitas ações
4. Usar `useRef` para elementos do DOM e valores que não redesenham
5. Aplicar `useMemo` e `useCallback` quando fizer sentido
6. Escrever **hooks personalizados** reutilizáveis

Antes de começar

- Aulas 05, 06 e 08 desta pasta

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 O problema: prop drilling

02 Context API

03 useReducer: estado com muitas ações

04 useRef: referências que não redesenham

05 useMemo e useCallback: memorização

06 Hooks personalizados

07 Depurando com React DevTools

08 Aprofundando: organizando o estado global

01

O problema: prop drilling

O problema: prop drilling

Imagine o usuário logado. O `App` sabe quem é, mas quem precisa mostrar o nome é o `Avatar`, quatro níveis abaixo. Sem outra ferramenta, cada componente do caminho recebe a prop só para repassá-la, mesmo sem usar. Isso se chama **prop drilling** (perfurar com props).

01

O problema: prop drilling

Aula 10 · Front-end

O problema: prop drilling



Três componentes recebendo uma prop que não usam.

O problema: prop drilling

Prop drilling

```
function App() {
  const [usuario, setUsuario] = useState({ nome: "Ana", tema: "escuro" });
  return <Layout usuario={usuario} />;
}
function Layout({ usuario }) { return <Cabecalho usuario={usuario} />; }
function Cabecalho({ usuario }) { return <Menu usuario={usuario} />; }
function Menu({ usuario }) { return <Avatar usuario={usuario} />; }
function Avatar({ usuario }) { return <span>{usuario.nome}</span>; }
```

02

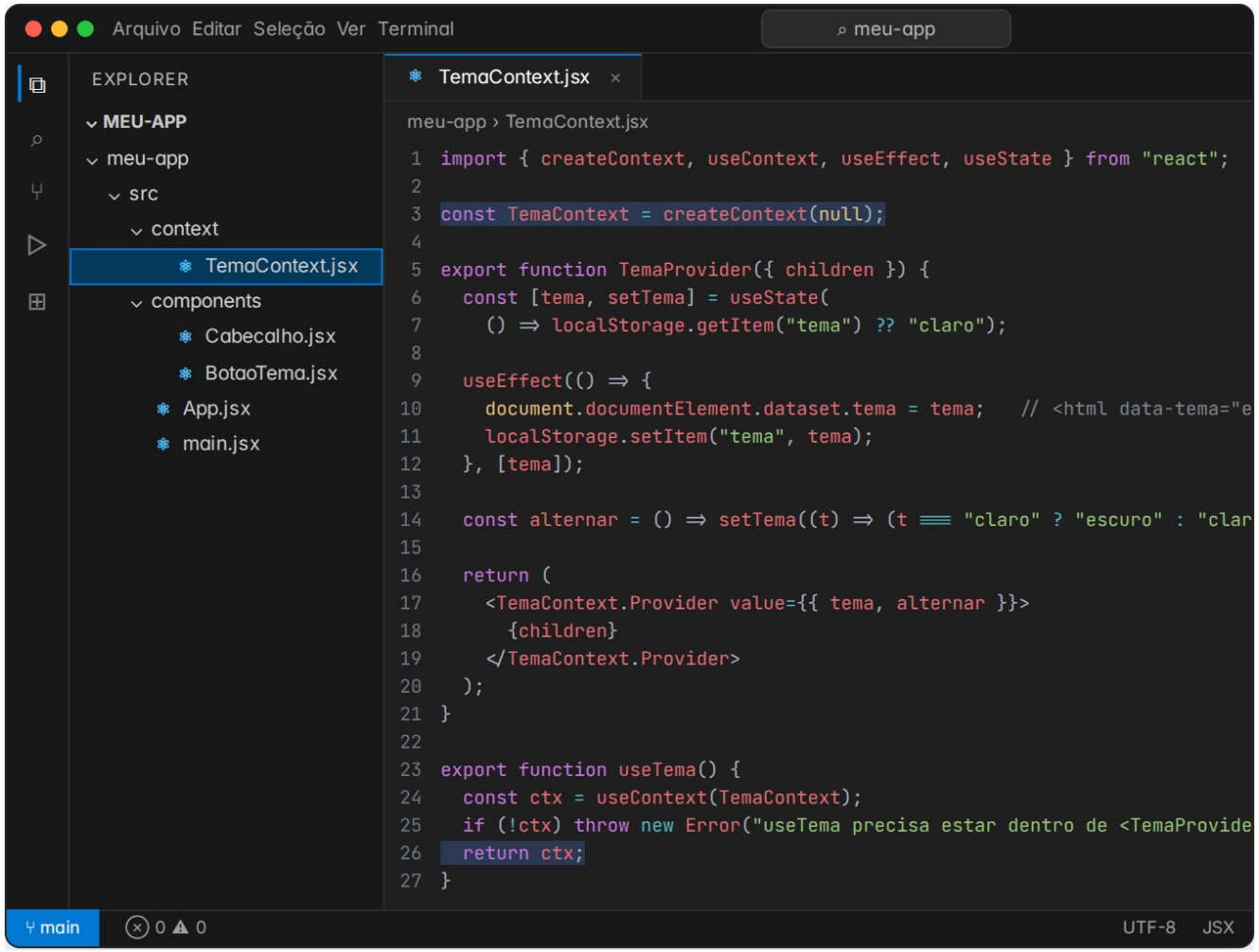
Context API

Context API

O **Context** cria um "canal" de dados: um componente **Provider** no alto da árvore oferece um valor, e qualquer componente abaixo dele lê esse valor com `useContext`, sem passar pelos intermediários.

Peça	Papel
`createContext(valorPadrao)`	Cria o canal
``	Oferece o valor para todos os componentes dentro dele
`useContext(MeuContext)`	Lê o valor no componente que precisa
Hook próprio (`useTema`)	Esconde o `useContext` e valida o uso fora do Provider

Context API



Contexto de tema claro/escuro: Provider + hook de acesso, no mesmo arquivo.

Context API

main.jsx

```jsx
import { TemaProvider } from "./context/TemaContext";

createRoot(document.getElementById("root"))
  .render(
    <StrictMode>
      <TemaProvider>
        <App />
      </TemaProvider>
    </StrictMode>,
  );
```

components/BotaoTema.jsx

```jsx
import { useTema } from "../context/TemaContext";

export default function BotaoTema() {
  const { tema, alternar } = useTema();
  return (
    <button onClick={alternar}>
      {tema === "claro" ? "🌙 Escuro" : "☀️ Claro"}
    </button>
  );
}
```

Context API

index.css: o tema troca as variáveis CSS

```css
:root { --fundo: #ffffff; --texto: #18212f; }
[data-tema="escuro"] { --fundo: #121417; --texto: #e8eaed; }

body { background: var(--fundo); color: var(--texto); }
```

Context API

Dica

O `AuthProvider` da pasta FullStack (aula 08) e o do projeto `SecureLoginPUC_React` seguem exatamente este formato: um Context com o usuário logado, `login` e `logout`, e um hook `useAuth()` para ler de qualquer componente.

Quando usar (e quando não usar) Context

- **Use** para dados que muitos componentes distantes precisam: usuário logado, tema, idioma, carrinho de compras.
- **Não use** para tudo: quando o valor do Provider muda, **todos** os componentes que leem aquele contexto redesenham. Estado que só um formulário usa continua local.
- Prefira **vários contextos pequenos** (`AuthContext`, `TemaContext`) a um contexto gigante com tudo.

02

Context API

Aula 10 · Front-end

03

useReducer: estado com muitas ações

useReducer: estado com muitas ações

Quando o estado tem várias formas de mudar (adicionar, remover, alterar quantidade, limpar), espalhar `setX` pelo componente fica confuso. O `useReducer` centraliza todas as mudanças em uma função pura, o **reducer**, que recebe o estado atual e uma **ação** e devolve o próximo estado.

03

useReducer: estado com muitas ações

Aula 10 · Front-end

useReducer: estado com muitas ações

reducers/carrinhoReducer.js

```
function carrinhoReducer(estado, acao) {
  switch (acao.type) {
    case "adicionar": {
      const existe = estado.find((i) => i.id === acao.produto.id);
      if (existe)
        return estado.map((i) => i.id === acao.produto.id ? { ...i, qtd: i.qtd + 1 } : i);
      return [...estado, { ...acao.produto, qtd: 1 }];
    }
    case "remover":
      return estado.filter((i) => i.id !== acao.id);
    case "alterarQtd":
      return estado.map((i) => (i.id === acao.id ? { ...i, qtd: acao.qtd } : i));
    case "limpar":
      return [];
    default:
      throw new Error(`Ação desconhecida: ${acao.type}`);
  }
}
```

useReducer: estado com muitas ações

Usando o reducer no componente

```
const [itens, dispatch] = useReducer(carrinhoReducer, []);

<button onClick={() => dispatch({ type: "adicionar", produto })}>Adicionar</button>
<button onClick={() => dispatch({ type: "remover", id: item.id })}>×</button>
<button onClick={() => dispatch({ type: "limpar" })}>Esvaziar carrinho</button>

const total = itens.reduce((soma, i) => soma + i.preco * i.qtd, 0);
```

useReducer: estado com muitas ações

	`useState`	`useReducer`
Bom para	Valores simples e independentes	Estado com várias ações relacionadas
Onde fica a lógica	Espalhada nos handlers	Concentrada no reducer
Testar	Junto com o componente	O reducer é uma função pura: teste sem React
Exemplo	Texto de um input, aberto/fechado	Carrinho, formulário em etapas, jogo

useReducer: estado com muitas ações

Observação

Combine os dois: um `CarrinhoProvider` que usa `useReducer` por dentro e expõe `itens`, `total` e `dispatch` pelo Context. É uma forma simples de “estado global” sem bibliotecas externas como Redux ou Zustand.

04

useRef: referências que não redesenham

useRef: referências que não redesenham

O `useRef` guarda um valor que **sobrevive entre renderizações** mas que, ao mudar, **não** faz o componente redesenhar. Tem dois usos principais:

Foco automático

```
// 1) acessar um elemento do DOM
const inputRef = useRef(null);

useEffect(() => {
  inputRef.current.focus();   // foca ao abrir
}, []);

<input ref={inputRef} placeholder="Buscar..." />
```

Timer e contador interno

```
// 2) guardar um valor sem redesenhar
const renderizacoes = useRef(0);
renderizacoes.current++;

const timerRef = useRef(null);
const iniciar = () => {
  timerRef.current = setInterval(tick, 1000);
};
const parar = () => clearInterval(timerRef.current);
```

useRef: referências que não redesenham

Atenção

Não leia nem escreva `ref.current` durante a renderização para decidir o que mostrar na tela: como mudar o ref não redesenha, a tela fica desatualizada. Para o que aparece na tela, use estado.

05

useMemo e useCallback: memorização

useMemo e useCallback: memorização

A cada renderização, todo o corpo do componente executa de novo. Normalmente isso é rápido. Quando um cálculo é **caro** (filtrar milhares de itens) ou quando uma função é passada para um filho memorizado, os hooks de memorização evitam trabalho repetido.

Hook	Memoriza	Exemplo
`useMemo(() => calculo, [deps])`	O **resultado** de um cálculo	Lista filtrada e ordenada
`useCallback(fn, [deps])`	A **própria função** (mesma referência)	Handler passado a um filho com `memo`
`memo(Componente)`	O componente: só redesenha se as props mudarem	Item de uma lista grande

useMemo e useCallback: memorização

Filtros da Pokédex com memorização

```
const filtrados = useMemo(() => {
  const termo = busca.toLowerCase();
  return pokemons
    .filter((p) => p.nome.toLowerCase().includes(termo))
    .filter((p) => !tipo || p.tipos.includes(tipo))
    .sort((a, b) => a.numero - b.numero);
}, [pokemons, busca, tipo]);      // recalcula só quando um destes muda

const handleFavoritar = useCallback((id) => {
  setFavoritos((f) => (f.includes(id) ? f.filter((x) => x !== id) : [...f, id]));
}, []);

const PokemonCard = memo(function PokemonCard({ pokemon, onFavoritar }) { ... });
```

useMemo e useCallback: memorização

✓ **Dica**

Não saia memorizando tudo: `useMemo` e `useCallback` também têm custo. Meça antes com a aba **Profiler** do React DevTools. Se nada está lento, o código simples é o melhor código. As versões recentes do React também têm o *React Compiler*, que pode memorizar automaticamente.

06

Hooks personalizados

Hooks personalizados

Um hook personalizado é uma função cujo nome começa com `use` e que usa outros hooks. Ele permite **reaproveitar lógica com estado** entre componentes. O projeto `Pokedex` tem quatro: `useFavoritos`, `usePokedexFilters`, `useSkin` e `useTheme`.

hooks/useBusca.js

```
import { useCallback, useEffect, useState } from "react";

export function useBusca(funcaoServico, ...args) {
  const [dados, setDados] = useState([]);
  const [carregando, setCarregando] = useState(true);
  const [erro, setErro] = useState("");

  const carregar = useCallback(async () => {
    setCarregando(true);
    setErro("");
    try {
      setDados(await funcaoServico(...args));
    } catch (e) {
      setErro(e.message);
    } finally {
      setCarregando(false);
    }
  }, [funcaoServico, ...args]);

  useEffect(() => { carregar(); }, [carregar]);

  return { dados, carregando, erro, recarregar: carregar };
}
```

Hooks personalizados

Usando o hook: o componente fica só com a tela

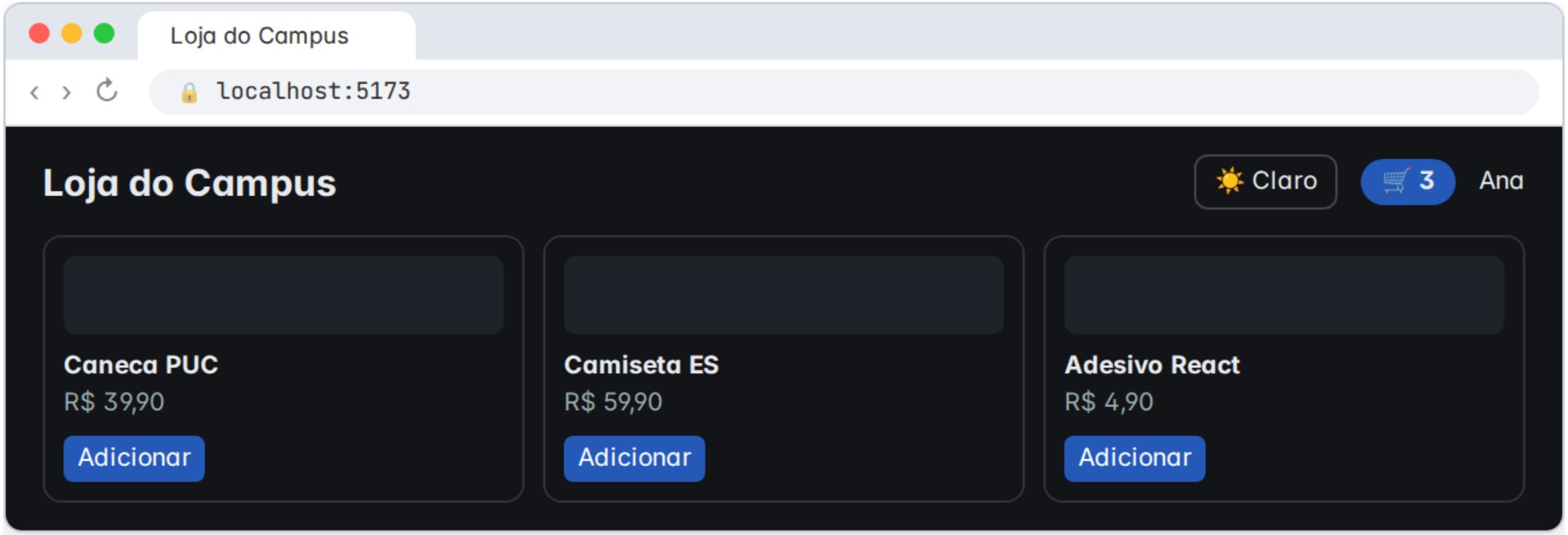
```
export default function ListaAlunos() {
  const { dados: alunos, carregando, erro, recarregar } = useBusca(listarAlunos);

  if (carregando) return <p>Carregando...</p>;
  if (erro) return <p>{erro} <button onClick={recarregar}>Tentar de novo</button></p>;
  return <ul>{alunos.map((a) => <li key={a.id}>{a.nome}</li>)}</ul>;
}
```

Hooks personalizados

Regras dos hooks	Por quê
Só chame hooks no **topo** do componente ou de outro hook	O React identifica cada hook pela ordem de chamada
Nunca dentro de `if`, `for` ou funções internas	A ordem mudaria entre renderizações
O nome do hook próprio começa com `use`	O linter e o React reconhecem e verificam as regras

Hooks personalizados



Exemplo integrando tudo: `TemaContext` (botão claro/escuro), `CarrinhoContext` com `useReducer` (contador 🛒) e `AuthContext` (nome do usuário).

07

Depurando com React DevTools

Depurando com React DevTools

1. Instale a extensão **React Developer Tools** no Chrome, Edge ou Firefox.
2. Abra o DevTools (`F12`) e vá à aba **Components**: a árvore mostra cada componente com props, estado, hooks e o valor de cada Context.
3. Clique em um componente e altere um valor do estado direto no painel para testar a tela.
4. Na aba **Profiler**, clique em gravar, use a tela e pare: o gráfico mostra quais componentes redesenharam e quanto tempo levaram.
5. Ative *Highlight updates when components render* para ver na tela, em tempo real, o que está redesenhando.

08

Aprofundando: organizando o estado global

Opções para estado compartilhado

useState + props

Poucos níveis de distância. Sempre a primeira opção.

Context + useReducer

Dados globais simples: usuário, tema, carrinho. Sem dependências.

Zustand / Redux Toolkit

Muitos dados globais que mudam com frequência; ferramentas de depuração.

TanStack Query

Dados que vêm do servidor: cache, recarga e estados de carregamento.

Aprofundando: organizando o estado global

context/CarrinhoContext.jsx: Context + reducer + persistência

```jsx
const CarrinhoContext = createContext(null);

export function CarrinhoProvider({ children }) {
  const [itens, dispatch] = useReducer(carrinhoReducer, [], () =>
    JSON.parse(localStorage.getItem("carrinho") ?? "[]"));

  useEffect(() => localStorage.setItem("carrinho", JSON.stringify(itens)), [itens]);

  const total = useMemo(() => itens.reduce((s, i) => s + i.preco * i.qtd, 0), [itens]);
  const valor = useMemo(() => ({ itens, total, dispatch }), [itens, total]);

  return <CarrinhoContext.Provider value={valor}>{children}</CarrinhoContext.Provider>;
}

export const useCarrinho = () => useContext(CarrinhoContext);
```

Aprofundando: organizando o estado global

O `useMemo` no `value` do Provider evita que todos os consumidores redesenhem quando o Provider redesenha por outro motivo: o objeto só muda quando `itens` ou `total` mudam.

08

Aprofundando: organizando o estado global

Aula 10 · Front-end

Context x biblioteca de estado

Context API

- Já vem no React
- Ótimo para valores que mudam pouco
- Todos os consumidores redesenham a cada mudança

vs

Zustand / Redux

- Dependência extra
- Componentes assinam só o pedaço que usam
- DevTools com histórico de ações

Aprofundando: organizando o estado global

Reducer testado sem React (Vitest)

```
import { describe, it, expect } from "vitest";
import { carrinhoReducer } from "./carrinhoReducer";

describe("carrinhoReducer", () => {
  it("soma a quantidade ao adicionar o mesmo produto", () => {
    const p = { id: 1, nome: "Caneca", preco: 39.9 };
    let estado = carrinhoReducer([], { type: "adicionar", produto: p });
    estado = carrinhoReducer(estado, { type: "adicionar", produto: p });
    expect(estado).toEqual([{ ...p, qtd: 2 }]);
  });
});
```

Aprofundando: organizando o estado global

Hook	Resumo em uma linha
`useState`	Valor que, ao mudar, redesenha o componente
`useEffect`	Efeito colateral depois da renderização
`useContext`	Lê o valor de um Provider acima
`useReducer`	Estado com ações centralizadas em uma função
`useRef`	Valor persistente que não redesenha; acesso ao DOM
`useMemo` / `useCallback`	Memoriza resultado / função
`useId`	Gera ids únicos para ligar `label` e `input`

Aprofundando: organizando o estado global

Na prática

Releia o `AuthProvider.jsx` do projeto `SecureLoginPUC_React` depois desta aula: ele combina Context, `useState`, `useEffect` e um hook de acesso. Tente reescrevê-lo com `useReducer` (ações `login`, `logout`, `erro`).

09

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

O que é prop drilling?

A Um erro de compilação

B Repassar props por componentes que não as usam

C Um hook do React

D Um tipo de rota

Resposta

O que é prop drilling?

A Um erro de compilação

✓ Repassar props por componentes que não as usam

C Um hook do React

D Um tipo de rota

Por quê: Context evita esse repasse.

Pergunta

Qual trio forma a Context API?

A useState, useEffect, useRef

B createContext, Provider e useContext

C Router, Route, Link

D fetch, then, catch

Resposta

Qual trio forma a Context API?

A. useState, useEffect, useRef

✓ createContext, Provider e useContext

C. Router, Route, Link

D. fetch, then, catch

Por quê: Normalmente embrulhado em um hook próprio como useTema().

Pergunta

Quando o `useReducer` é mais indicado que o `useState`?

A Para um único booleano

B Para estados com várias ações relacionadas, como um carrinho

C Nunca

D Só com TypeScript

Resposta

Quando o `useReducer` é mais indicado que o `useState`?

A Para um único booleano

✓ Para estados com várias ações relacionadas, como um carrinho

C Nunca

D Só com TypeScript

Por quê: O reducer concentra a lógica em uma função pura e testável.

Pergunta

O que acontece ao alterar `ref.current`?

A. O componente redesenha

B. O valor muda, mas o componente não redesenha

C. Dá erro

D. O estado é apagado

Resposta

O que acontece ao alterar `ref.current`?

A. O componente redesenha

✓ O valor muda, mas o componente não redesenha

C. Dá erro

D. O estado é apagado

Por quê: Para o que aparece na tela, use estado.

Pergunta

Qual a regra dos hooks?

A. Podem ser chamados dentro de if

B. Só no topo do componente ou de outro hook

C. Só em classes

D. Só no main.jsx

Resposta

Qual a regra dos hooks?

A. Podem ser chamados dentro de if

✓ Só no topo do componente ou de outro hook

C. Só em classes

D. Só no main.jsx

Por quê: O React identifica os hooks pela ordem de chamada.

Fechamento

Resumo da aula

- ✓ **Prop drilling** é repassar props por componentes que não as usam; Context resolve.
- ✓ Context = `createContext` + `Provider` + `useContext`, embrulhado em um hook próprio (`useTema`).
- ✓ `useReducer` centraliza estados com várias ações em uma função pura e testável.
- ✓ `useRef` acessa o DOM e guarda valores que não redesenham.
- ✓ `useMemo`/`useCallback`/`memo` só quando houver custo medido; hooks próprios reaproveitam lógica.

Exercícios

1. Crie um `TemaContext` com claro/escuro salvo no `localStorage` e aplique em um projeto seu.

2. Implemente o carrinho com `useReducer` (adicionar, remover, alterar quantidade, limpar) e o total.

3. Escreva o hook `useBusca` e use-o em duas telas diferentes.

4. Use o Profiler do React DevTools para encontrar um componente que redesenha sem necessidade e corrija com `memo`.

Para ir além

- react.dev/learn/passing-data-deeply-with-context
- react.dev/reference/react/useReducer
- react.dev/learn/reusing-logic-with-custom-hooks
- Projetos `Pokedex` (hooks) e `SecureLoginPUC_React` (AuthProvider)

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 10?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web